GitHub Copilot Workshop Day2

Agenda

- Agent Extensibility Overview
- Agent Hooks
- MCP 介紹
- Build MCP Server with .NET 10
- Custom Agent + Skills + MCP
- Build Custom Agent Plugin

Agent Extensibility Overview

GitHub Copilot Customization features

官方說明:

GitHub Copilot Customization features

名稱	可客製化
Custom instructions	✅
Agent skills	✅
Tools	❌
MCP Server	✅
Hooks	✅
SubAgent	❌
Custom Agents	✅
Extensions	✅
Plugins	✅

其中 Extensions 需撰寫 JavaScript 程式且只能安裝在 GitHub Copilot app & GitHub Copilot CLI，其他可客製化的透過撰寫相關Prompt或設定檔

Custom instructions

https://docs.github.com/en/copilot/concepts/customizations/custom-instructions

自訂指令（Custom instructions）讓使用者能夠為 GitHub Copilot 提供具體的指導，以規範其運作行為。這可以包含對程式碼風格、命名慣例或其他專案特定需求的偏好設定。



- GitHub Copilot Custom Instructions 可以分為專案全域與針對特定檔案類兩種：

  專案全域作用：

  ```
  .github/
  └─ copilot-instructions.md
  ```

  針對特定檔案類型：

  ```
  .github/instructions/abcd/
                      └─ abcd.instructions.md
  .github/instructions/defg/
                      └─ defg.instructions.md
  ```

- 全域 Custom Instructions 可在 VSCode Chat View 的 GitHub Copilot 或 GitHub Copilot CLI 下 `/init` 指令建立。

- 建議的範圍區分：
  全域 copilot-instructions.md 負責專案規則，公訂標準
  『AGENTS.md(https://agents.md/)』 負責 Agent 行為

Agent skills

https://docs.github.com/en/copilot/concepts/customizations/agent-skills
代理技能（Agent skills）允許使用者為 GitHub Copilot 定義特定的功能或行為。這些技能可以擴展 Copilot 的能力，使其能夠執行特定任務或與其他工具整合。



- GH Copilot 將放在 ***.github/skills/*** 或 ***.agent/skills/*** 目錄下，每個 Skill 對應一個自命名子目錄，內含 SKILL.md 定義markdown檔案的目錄結構，視為一個 Agent skill。

  ```
  .github/skills/
    └ my-skill/
        ├ SKILL.md
        └ ...
  ```

  或在 .agent/skills/ 目錄下：

  ```
  .agent/skills/
    └ my-skill/
        ├ SKILL.md
        └ ...
  ```

  遵循 Agent skill 的公訂規範(https://agentskills.io/specification)即可使用
- GH Copilot 使用 “slash command” (**`/skill名稱`**) 來明確指定LLM去讀哪個 Agent skill 來使用。
- 另有一個 `/skills` 指令，列出目前所有可用 Agent skills。

GH Copilot 在 IDE (Visual Studio, VSCode, Xcode, JetBrains IDE) 中還有支援一個舊版的『Prompt files』功能，但 *建議將其移轉至 Agent skill*。

MCP Server

https://docs.github.com/en/copilot/concepts/customizations/mcp

MCP Server 為 GitHub Copilot 提供擴展功能的伺服器端組件，使用者可以MCP Server 中定義自訂行為、整合外部服務，並與 Copilot 進行互動。

- 在VSCode中使用MCP Server要在設定檔裡進行相關設定:
  - **.vscode/mcp.json** (舊版設定檔)
  - **.mcp.json** (新版設定檔)
- MCP Server 的設定通常包含伺服器端點、認證資訊以及其他與 Copilot 互動所需的參數。
- 目前支援的 MCP Server 有分為 stdio 和 http 兩種。
- GH Copilot 目前僅能使用 MCP Server 提供的 『Tools』 功能。

Hooks

https://docs.github.com/en/copilot/concepts/customizations/hooks

Hooks 允許使用者在特定事件發生時，執行自訂的程式碼或行為。這使 GitHub Copilot 跑的代理能夠在不同的操作流程階段，觸發使用者定義的邏輯，實現更靈活的擴展。



- Hooks 設定檔位置：
  - **.github/hooks/ * .json** (VSCode, GitHub Copilot CLI, Copilot cloud agent)
  - **~/.copilot/hooks/ * .json** (VSCode, GitHub Copilot CLI)
  - **~/.copilot/settings.json** (GitHub Copilot CLI)
- Hooks 非公訂標準，在 VSCode 還是預覽功能，尚無法與 Claude Code 的 Hooks 設定檔共用

Custom Agents

https://docs.github.com/en/copilot/concepts/customizations/custom-agents

自訂代理（Custom Agents）允許使用者建立專屬的 GitHub Copilot 代理，以實現特定的功能或行為。這使得使用者能夠根據自身需求，擴展 Copilot 的能力。

> - Custom Agent 設定檔位置為 **.github/agents/ * .md**
> - Custom Agent 設定檔架構類似 Agent skill，也是含有 YAMLForntmatter 的 markdown 格式檔案
> - 可用Prompt Engineering中的『角色扮演』技巧來控制 Custom Agent 行為
> - 可額外記載允許使用的 LLM Model、 Tools、 MCP Server資訊

切莫和公訂標準『AGENTS.md (https://agents.md/)』混淆，這是類似 Custom instructions：

AGENTS.md and GitHub Copilot: How Copilot Reads It, What It Ignores, and How to Make It Work

Custom Agent 範例

Plugins

https://docs.github.com/en/copilot/concepts/customizations/plugins
插件（Plugins）允許使用者為 GitHub Copilot 增加額外的功能或整合。透過撰寫插件，使用者可以擴展 Copilot 的能力，並與其他工具或服務進行互動。



- 插件的設定檔架構為 JSON 格式，記錄有：
  - MCP Server config
  - Custom Agent
  - Agent Skill
  - Automation Templates (VSCode only)
  - Hook (not public standard)
  - LSP server config (not public standard)
- Plugin放在同電腦檔案目錄或是在 git repo 上的都可當安裝源

Plugin 遵循『Agent Plugins(https://agent-plugins.org/)』這個公訂標準，所以一些 Claude Code Plugin 也可使用，但支援程度請見官方文件

使用時機考量

- 從作用範圍/是否為公訂規格的角度來分類：
  - Custom Instructions – Copilot-instruction.md / AGENTS.md
  - Custom Agents – ***AgentName***.agent.md
  - Case-by-case – Prompt file / Agent Skill
  - Event-based – Hooks
    (VSCode支援格式和專門給GH Copilot CLI/Copilot Cloud Agent用Hook格式有所不同)
  - Package/Distribute – Agent Plugin
- 官方建議分類：GitHub Copilot Customization Handbook

Agent Hooks

Hooks

- 官方說明:
  - Configure agent hooks in Visual Studio Code (Preview)
  - Introduction to hooks
  - Hooks for GitHub Copilot
- 在 AI Agent 執行工作流程到不同狀態時會觸發的Shell Script，目前支援 Bash & PowerShell script （類似 Git Hooks 概念）
- 雖然其他 Coding Agent 如 Claude Code 也有Hook，但事件觸發機制名稱不同；且 GitHub Copilot CLI 支援的 Hook 事件種類比 VSCode GH Copilot 來得多(又是另一種版本的Hook)
- 在VSCode中，將設定檔(hook.json)以及配套呼叫的script檔案目錄等拷貝到 **.github/hooks** 目錄內即發揮作用
- User Level Hook 則根據作業系統不同而存放在不同的目錄，例如:
  - macOS / Linux: **~/.copilot/hooks**
  - Windows: **%USERPROFILE%\.copilot\hooks**

Hooks 事件適用範疇

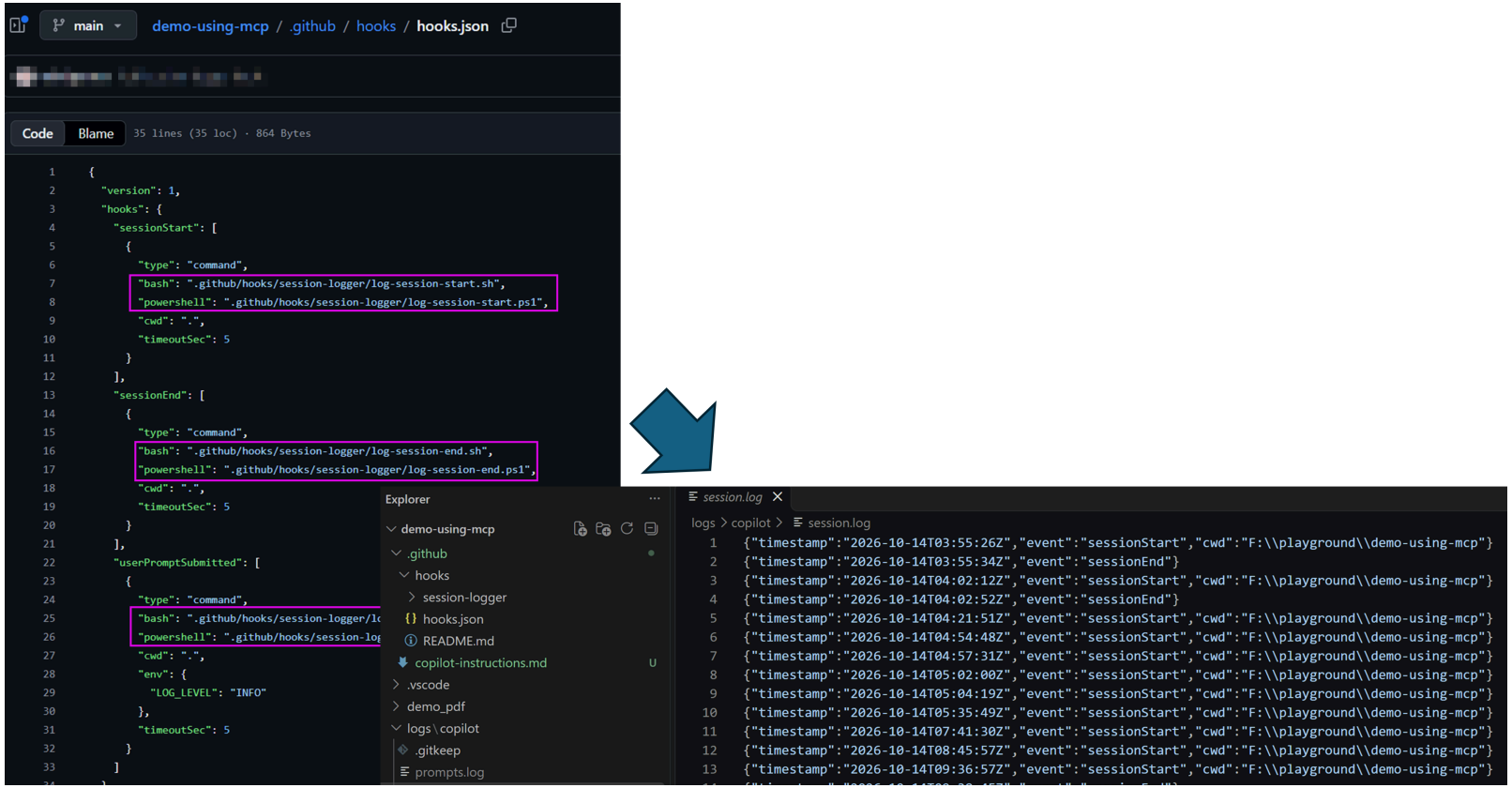
GitHub Copilot CLI Hooks 種類

事件名稱	說明	觸發時機	用途
`sessionStart`	Agent Session 啟動	於 Session 開始時	注入初始log記錄
`userPromptSubmitted`	使用者送出 Prompt	使用者提交 Prompt，LLM回應前	記錄使用者輸入的 Prompt
`preToolUse`	Tool 執行前	Agent 要呼叫任何Tool之前	審核或記錄將使用之Tool
`postToolUse`	成功的 Tool 執行後	Tooling 回傳成功	格式化回傳結果
`errorOccurred`	Tool 執行失敗或是有其他系統錯誤	Tooling 回傳失敗、拋出錯誤訊息	記錄錯誤訊息，便於追蹤和偵錯
`agentStop`	Agent 停止	主 Agent 完成	驗證最終結果

Lab6: Session Logger hook

Lab6 GitHub 詳細步驟

範例執行畫面：



Hooks 範例

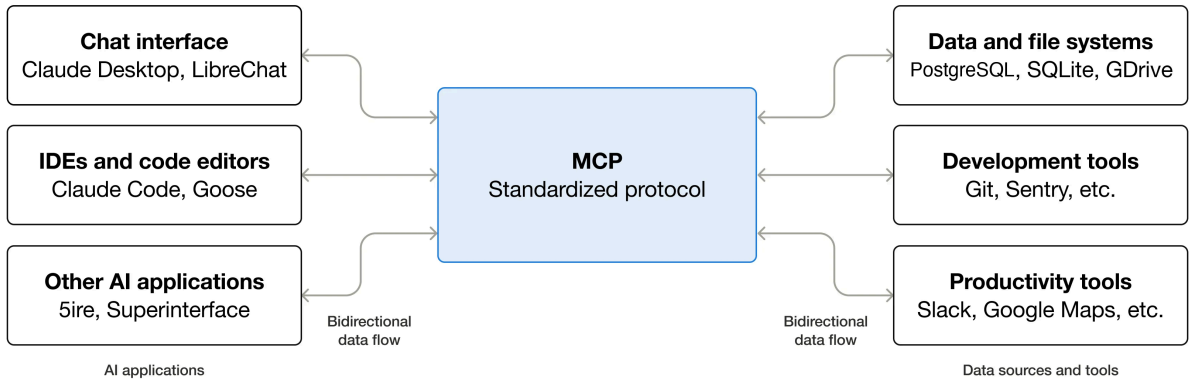
- Awesome GitHub Copilot Hooks 範例
- Agent Hooks Demo

MCP 介紹

- MCP 簡介
- VSCode 使用 MCP (MSLearn、GitHub、MarkItDown)

MCP 簡介

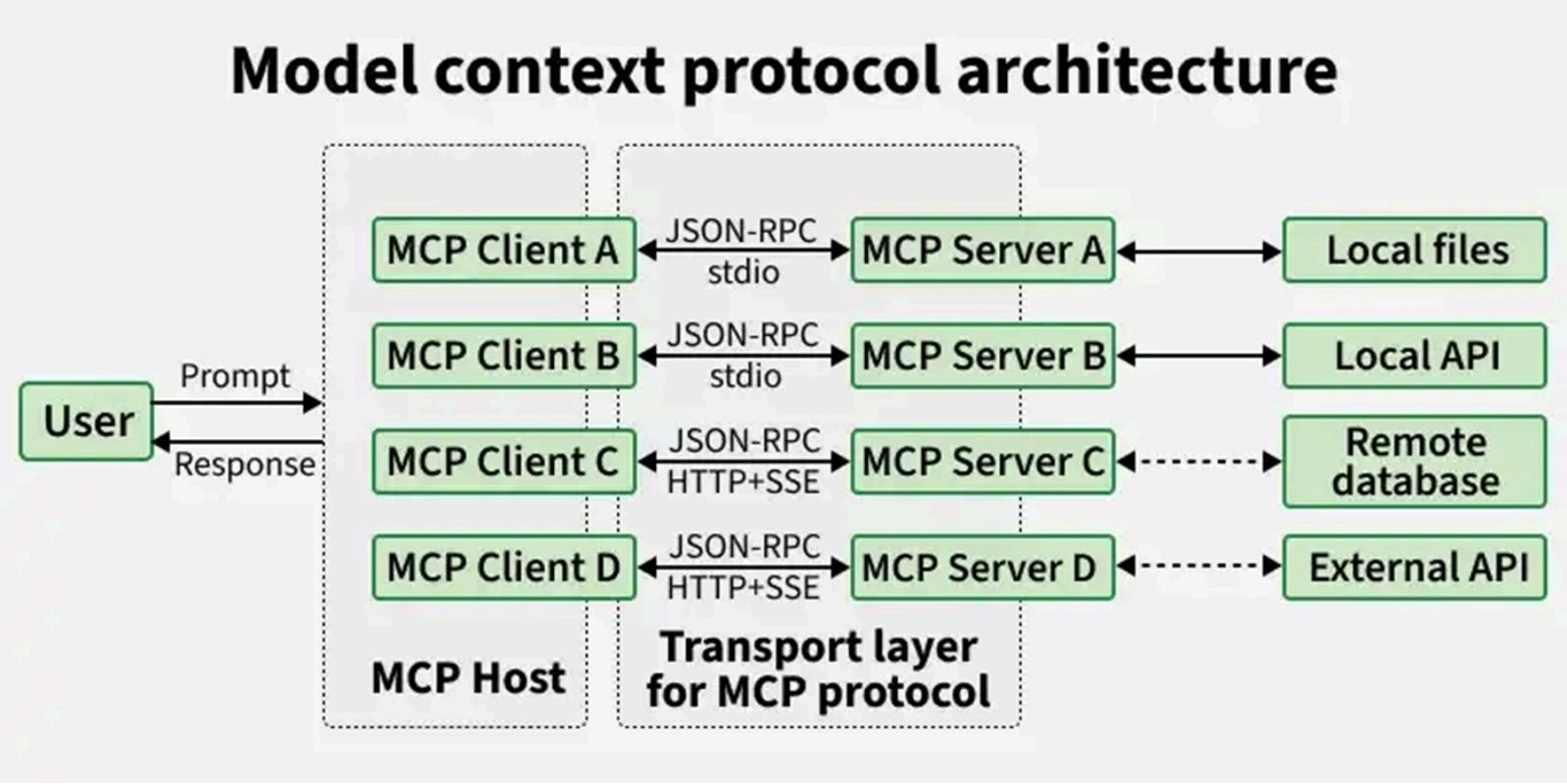
- 整合 AI Agent 各種外部工具呼叫的通訊協定(JSON-RPC)
  https://modelcontextprotocol.io/
  https://github.com/modelcontextprotocol
- Ver 2.0 重新定為 State-less 的通訊協定



- MCP讓LLM突破原本只有查詢/回覆文字輸出的限制，開始做事
- MCP Server藉由串接既有REST API，讓AI Agent跟既有系統互動

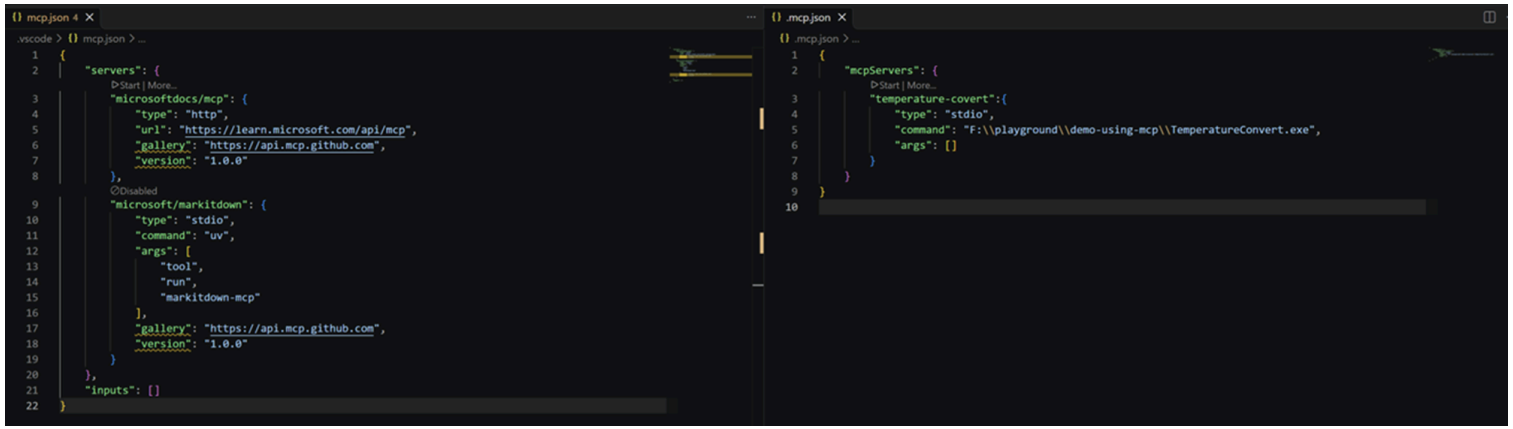
MCP 術語與架構

- 實務上會由 Agent Harness 框架擔任 MCP Host 的角色，讓承載的 LLM 做為 MCP Client來使用 MCP 呼叫
- Ver2.0的MCP Server底層實際通訊機制為本機端(stdio)以及遠端(http, streamable-http), SSE deprecated



VSCode 使用 MCP

- 在VSCode上使用MCP Server，其實就是在設定檔裡添加設定: https://code.visualstudio.com/docs/agent-customization/mcp-servers
- 設定檔有新舊兩種版本，檔案位置以及格式不同(建議往新版本升級)
  - Old(VSCode only): `.vscode/mcp.json`
  - New(Portable): `.mcp.json`



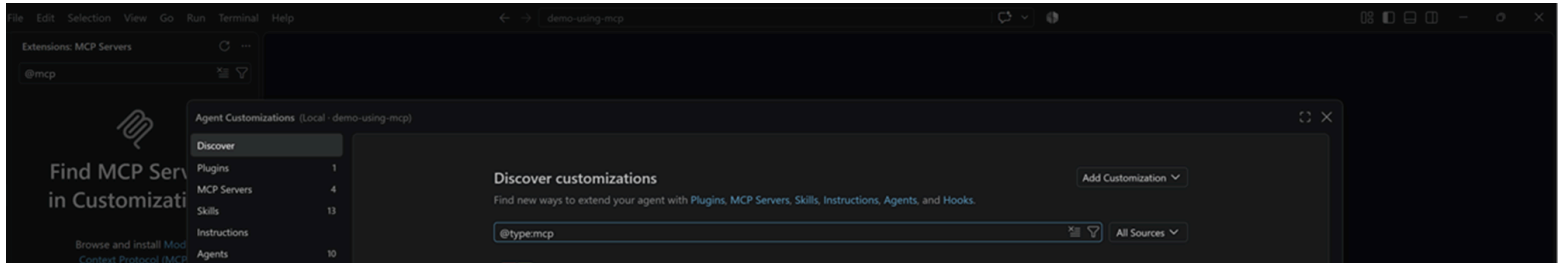
- 使用目前VSCode的UI功能設定時，在專案中沒有舊版設定檔時，優先建立新版設定檔
- 除了專案範圍的設定檔之外，還可以有使用者範圍的設定檔，通常位於使用者主目錄下的 `.mcp.json`，或是設定 `COPILOT_Home` 環境變數指定位

VSCode 使用 MCP Marketplace 安裝 MCP Server

1. 快速鍵 Ctrl(Cmd)+Shift+P 搜尋 MCP:

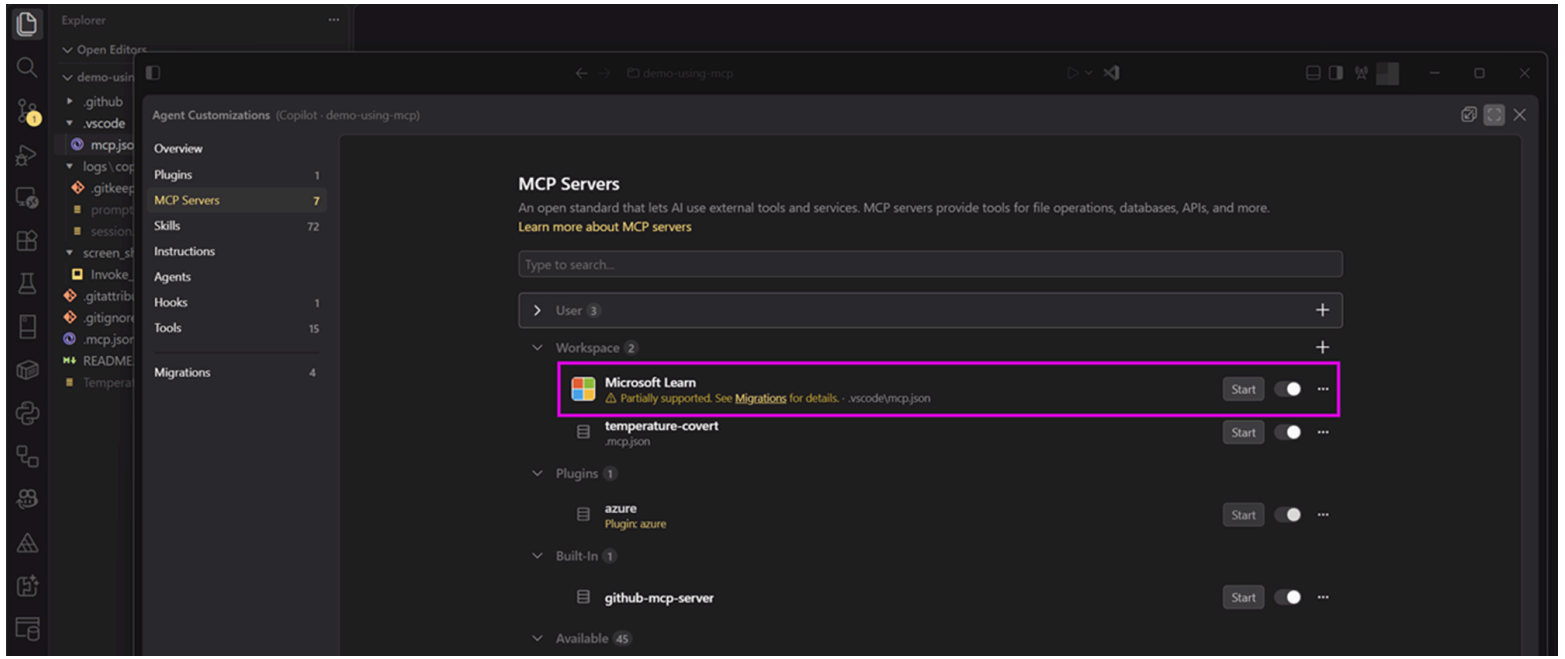


2. 導引至 VSCode Agent Window 的 Agent Customization 設定頁面，選擇 MCP Servers 進行安裝與設定:



VSCode 設定 MS Learn MCP Server

- MS Learn MCP Server 可讓 Agent 查找 MS Learn 網站文章/範例程式碼 https://learn.microsoft.com/training/support/mcp
- 假如先前已經隨 VSCCode 早期版本內建而安裝過，在 Agent Customization 設定頁面會額外顯示可點選進行『Migrations』超連結：

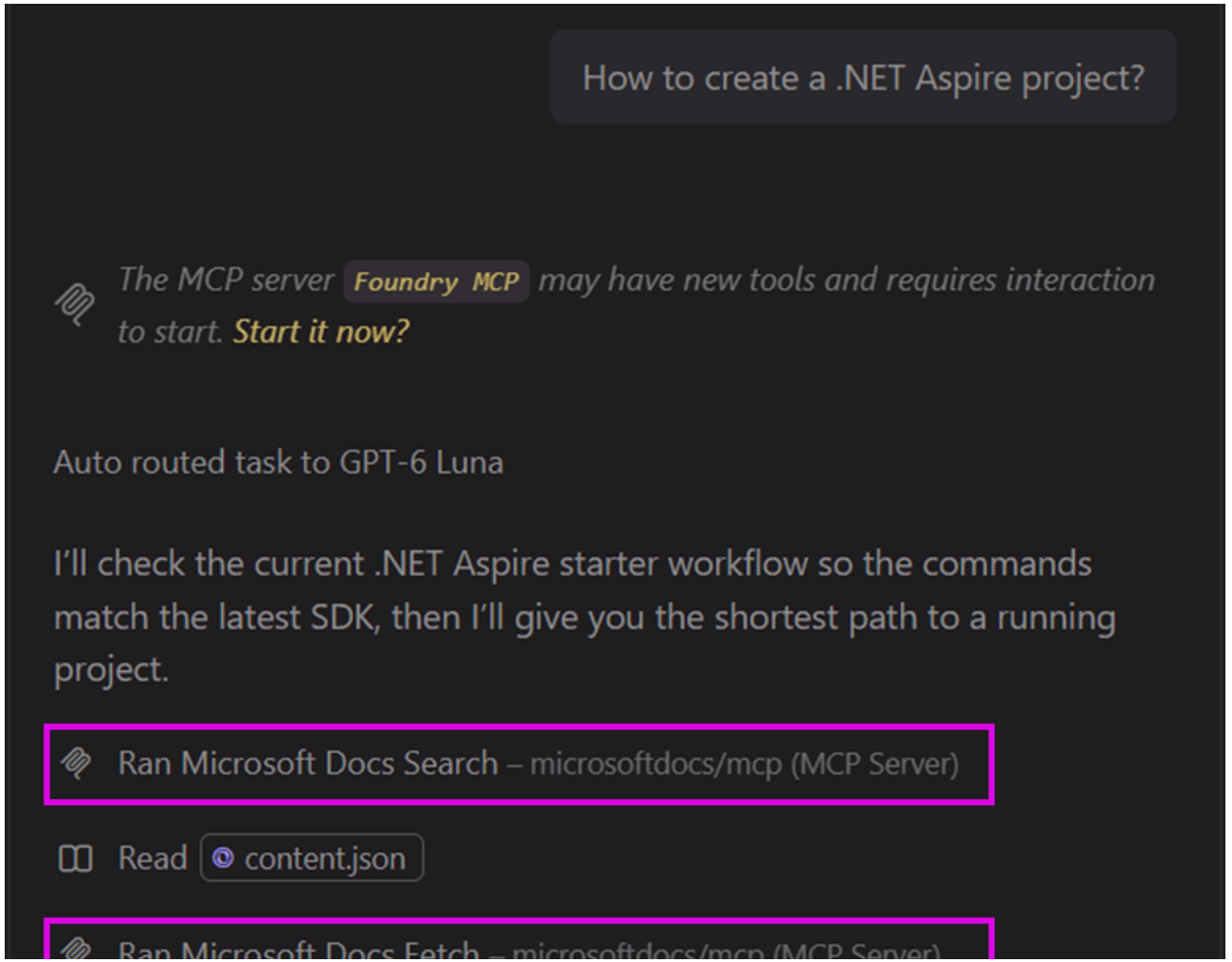


安裝完成之後應該會有如下的 json 設定：

```
1 {
```

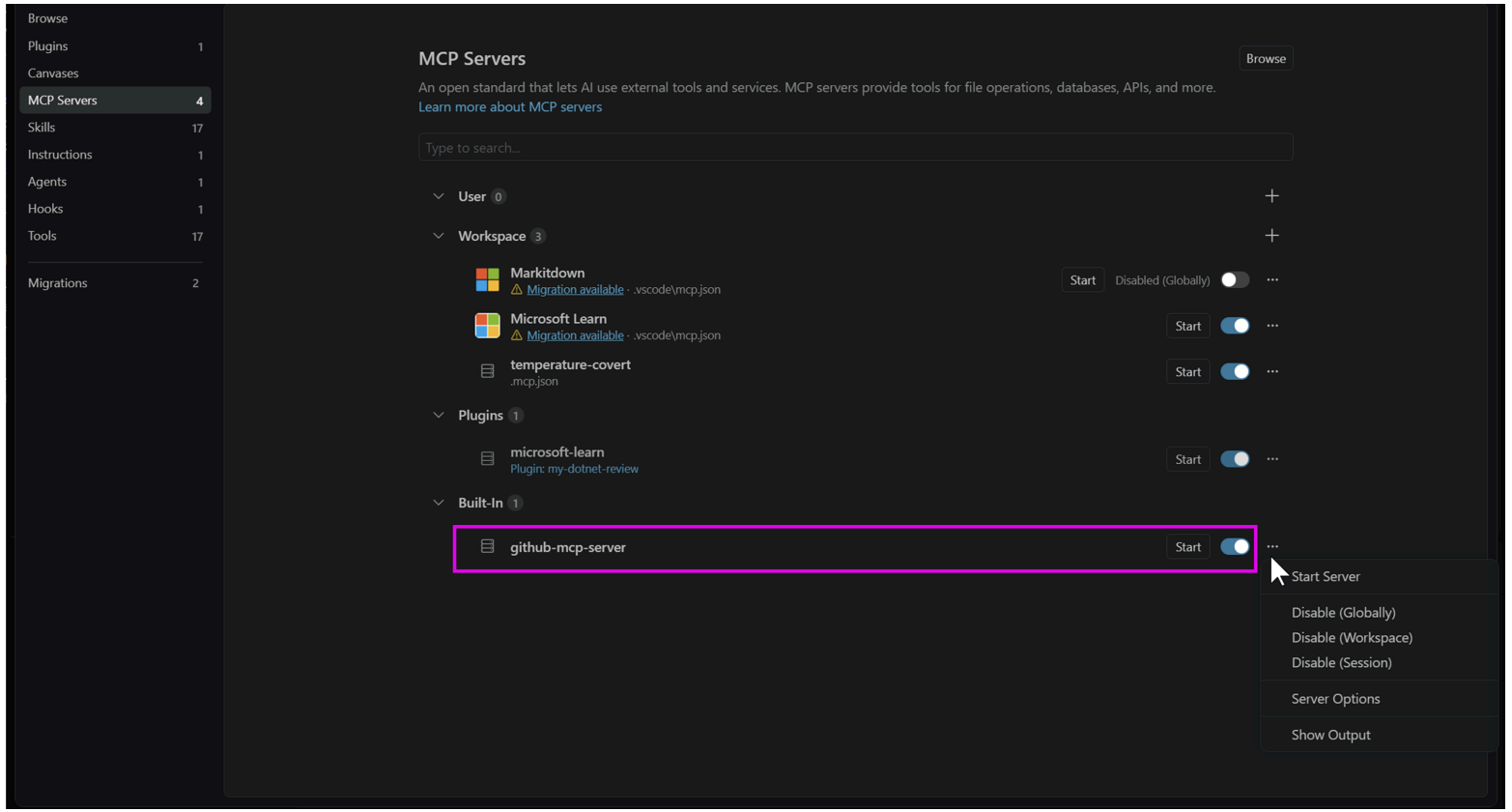
VSCode 使用 MS Learn MCP Server 範例

在跟 AI 對話時如果有些問題 Agent 使用的LLM模型不懂，會嘗試使用 MS Learn MCP Server獲取資訊：



VSCode 設定 GitHub MCP Server

- GitHub MCP Server 可對特定 GitHub repository做各種動作: https://github.com/github/github-mcp-server
- 新版的 VSCode 直接內建 GitHub MCP Server， 無需額外安裝:



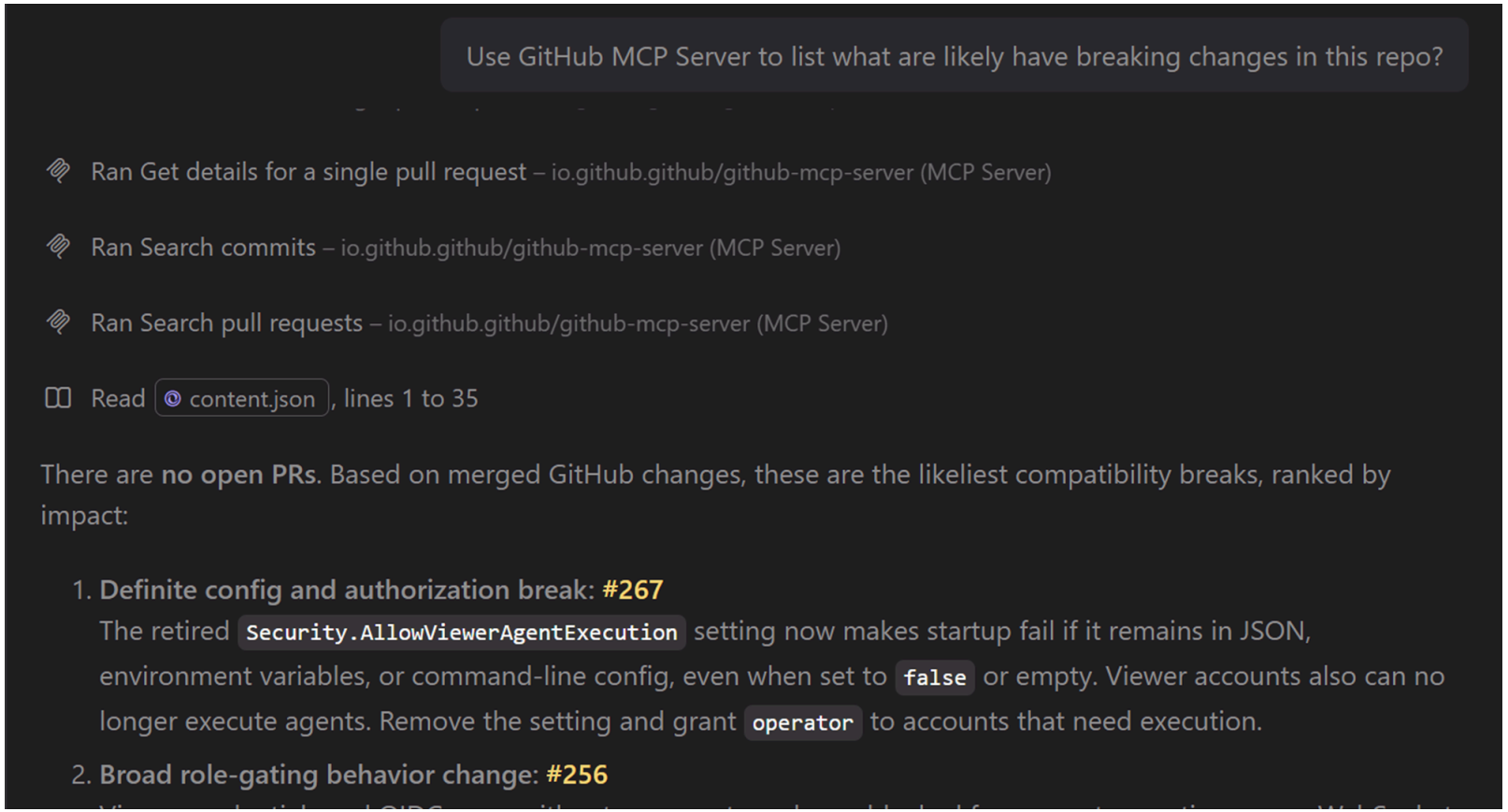
- 也可以直接手動改設定檔，使用 OAuth 進行認證:

VSCode 使用 GitHub MCP Server

第一次使用時會提示並引導開啟網頁做OAuth登入

之後就可以直接使用，例如 git pull 再用VSCode開啟OpenClaw.NET 的 GitHub 專案後，直接問 AI 得知是否有哪些破壞性版本變更：

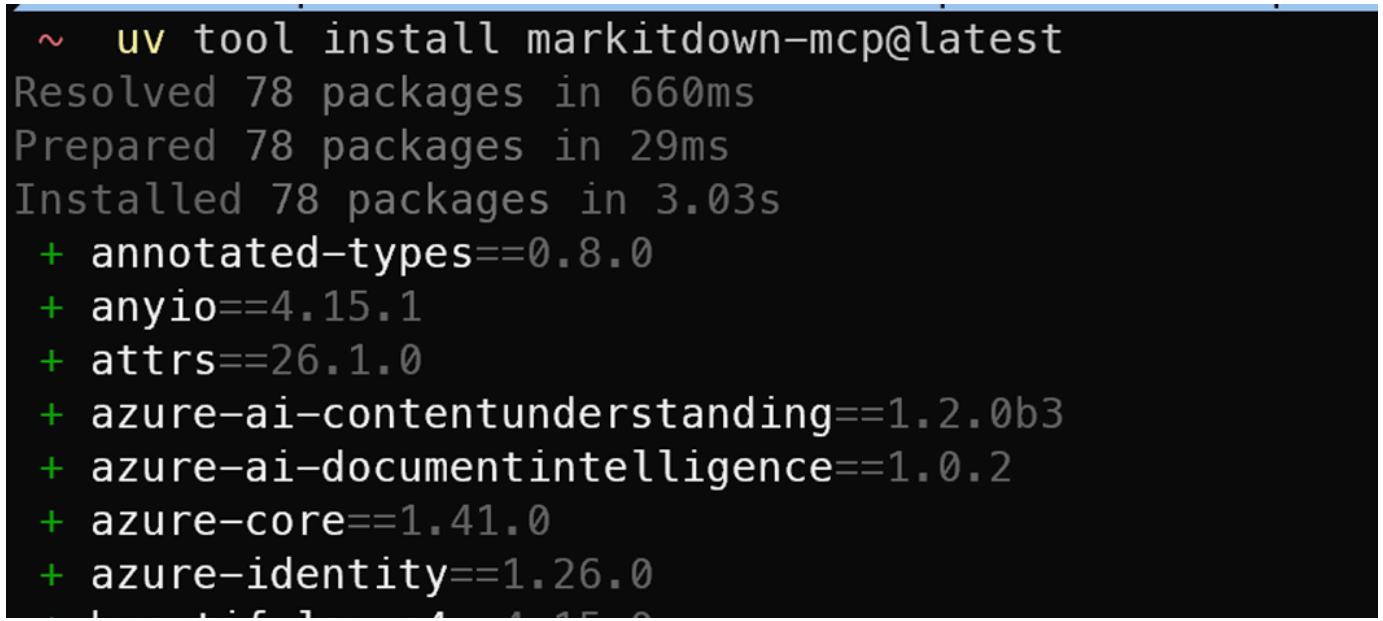
```
1  Use GitHub MCP Server to list what are likely have breaking changes in this repo?
```



VSCode 設定 MarkItDown MCP Server

- MarkItDown MCP Server: https://github.com/mcp/microsoft/markitdown
- 可用來擷取PDF/docx/xlsx檔案內容並轉換為Markdown格式內容，便於後續LLM AI使用
- 這是 stdio 模式的 MCP Server，建議設定前先安裝 uv 命令列之後，再由 uv 安裝 markitdown-mcp 全域命令列

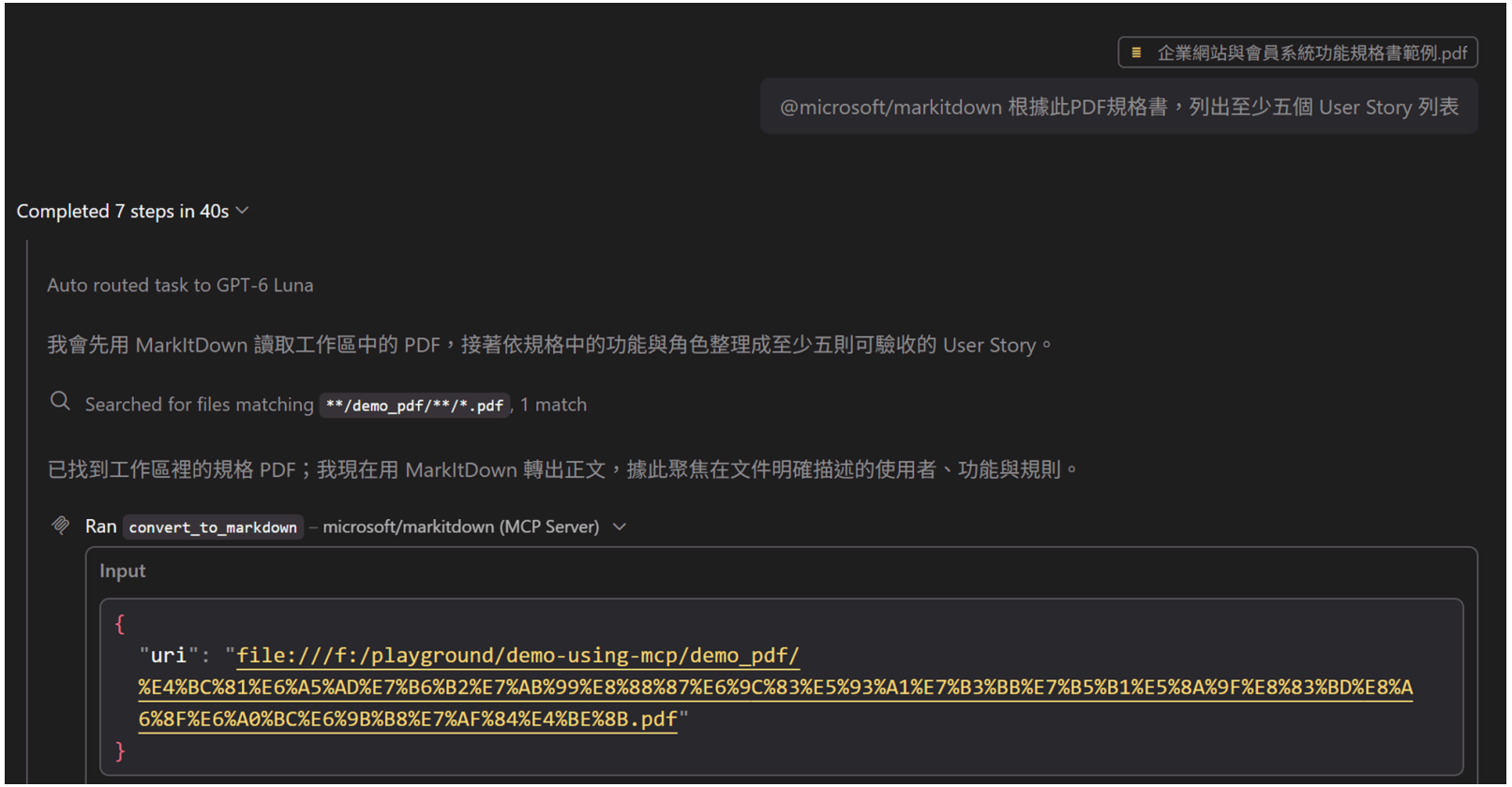
```
uv tool install markitdown-mcp@latest
```



Lab7: VSCode 使用 MarkItDown MCP Server

Lab7 GitHub 詳細步驟

範例執行畫面:



Build MCP Server with .NET 10

自製地端MCP Server

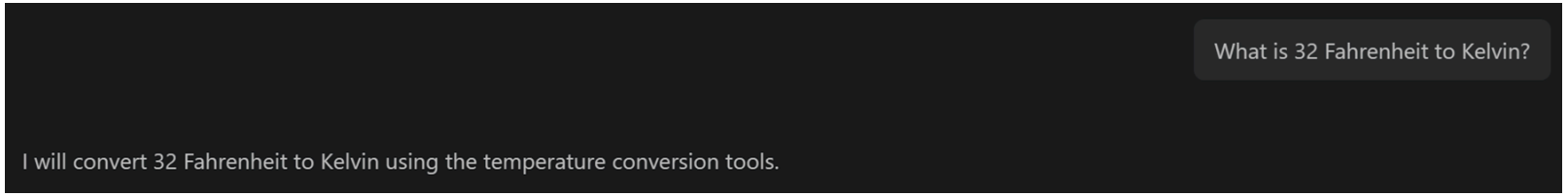
- MCP C# SDK 最主要是提供了MCP endpoint的 C# Attribute，套用在C#類別 & 方法程式碼上
- 在 C#方法上Attribute 的Description敘述為 LLM 查找/使用此方法的依據，需確實完善填寫，避免語意模糊或多個方法相似
- 在 C#類別的Attribute為便於 hosting程式碼註冊MCP端點使用，如果是手動註冊該C#類別為MCP端點，就可不必套用該Attribute

Lab8 自製地端MCP Server

使用 MCP C# SDK 建立命令列程式然後設定為地端 MCP Server

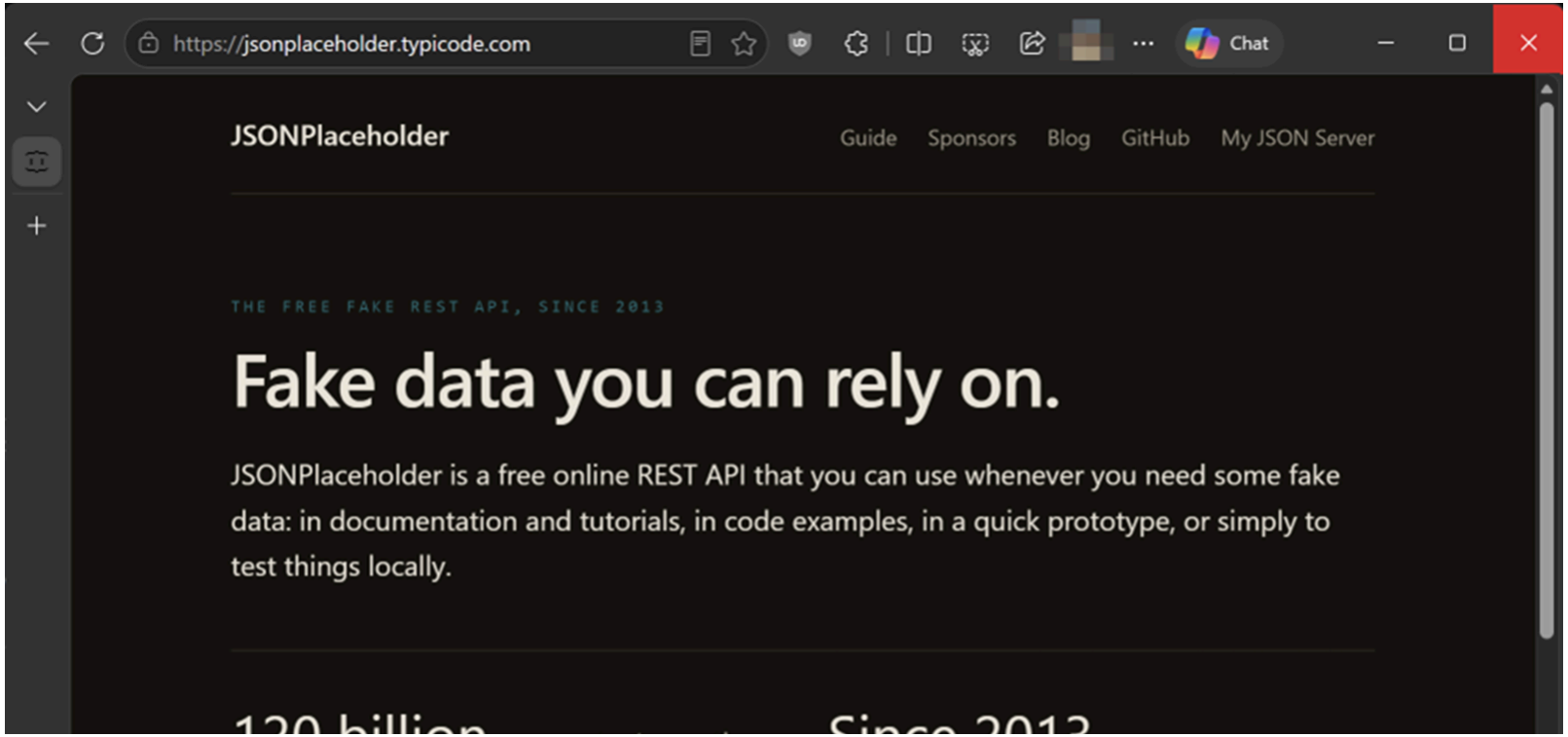
- 先建立一個新的 .NET 10 命令列專案
- 安裝 MCP C# SDK 套件
- 實作 MCP Server 的邏輯
- 在 VSCode 設定中加入此地端 MCP Server 的資訊

1. 溫度轉換範例程式：
   https://github.com/dotnet-mcp-lab/CSharpMcpDemo/tree/main/MCPServer/TemperatureConvert
2. 使用 .NET SDK 編譯成單一命令列程式之後，複製到 demo-using-mcp 專案內使用：
   https://github.com/dotnet-mcp-lab/demo-using-mcp



Lab9 自製雲端MCP Server (ASP.NET Core)

- 示範如何串接現有的REST API (JSONPlaceholder)轉換為MCP Server 範例程式碼
- 為 ASP.NET Core minimal API 專案裝 ModelContextProtocol.AspNetCore 這個 MCP SDK Nuget 套件: Nuget 套件



Lab10 自製雲端MCP Server (Azure Function)

- Azure Function版的串接JSONPlaceholder轉換為MCP Server
  範例程式碼
- 使用 Azure Function SDK 的 MCPToolTrigger實作MCP endpoint:
  教學文件
- Azure Function啟動後會有對應的 Tool Trigger 可供LLM AI呼叫

在 VSCode 內執行起來會列出可呼叫的 MCP Tool Trigger 列表

```
[2026-10-13T11:31:41.410Z] Found F:\source\CSharpMcpDemo\MCPServer\JsonPlaceHolder\AzFunc.JsonPlaceHolderMCP\AzFunc.JsonPlaceHolderMCP.csproj
. Using for user secrets file configuration.
[2026-10-13T11:31:43.317Z] MCP server endpoint: http://localhost:7071/runtime/webhooks/mcp
[2026-10-13T11:31:43.317Z] MCP server legacy SSE endpoint: http://localhost:7071/runtime/webhooks/mcp/sse
[2026-10-13T11:31:46.660Z] Azure Functions .NET Worker (PID: 84504) initialized in debug mode. Waiting for debugger to attach...
[2026-10-13T11:31:46.695Z] Worker process started and initialized.

Functions:

        GetTodosById: mcpToolTrigger

        GetTodosByUserId: mcpToolTrigger

        GetUserById: mcpToolTrigger

        ListTodos: mcpToolTrigger

        ListUsers: mcpToolTrigger

For detailed output, run func with --verbose flag.
[2026-10-13T11:31:51.713Z] Host lock lease acquired by instance ID '00000000000000000000000023975A74'.
```

Debug MCP Server

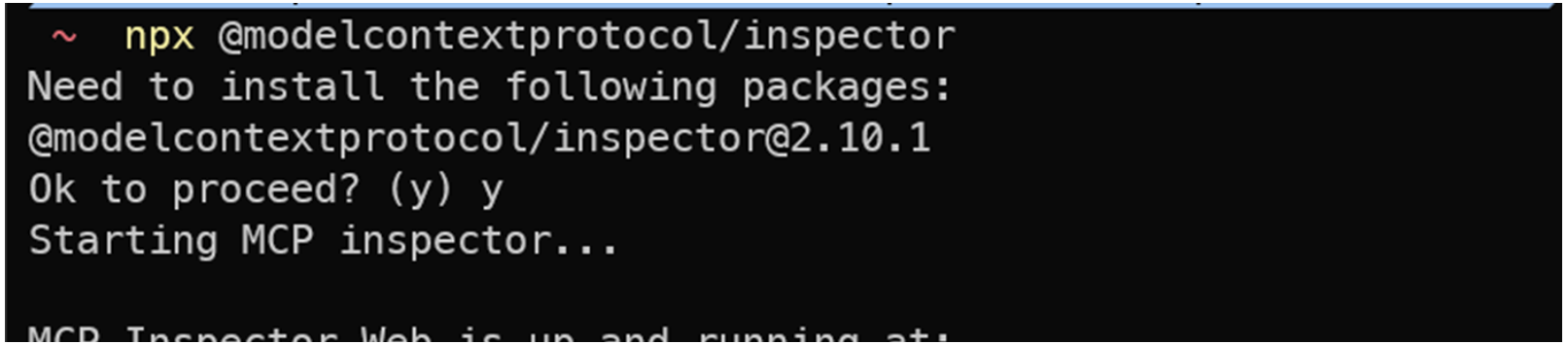
除了直接設定 VSCode Coding Agent 串MCP Server之外，也可用不消耗token的方式驗證 MCP Server：

- MCP Inspector(https://github.com/modelcontextprotocol/inspector)
  需安裝Node.js v22以上版本
- POSTMan App (https://www.postman.com/downloads/)

以下示範使用 MCP Inspector 接 ASP.NET Core 的 MCP Server 使用命令列啟動網頁UI

```
1 npx @modelcontextprotocol/inspector
```

啟動後會在瀏覽器中開啟 MCP Inspector 的網頁介面，並列出可呼叫的 MCP Tool 列表



Custom Agent + Skills + MCP

Lab11: 結合 Skills + MCP 製作客製化 Agent

Lab11 操作步驟及範例原始碼

介紹 MCP 這個可以和外部工具溝通的媒介之後，我們可以將 Skills 與 MCP 結合，製作出更強大的客製化 Agent，讓 AI 能夠不僅查詢資訊，還能執行各種操作。

以下是結合 Skills 與 MCP 的客製化 .NET Backend Developer Agent 架構示意圖：

```
.NET Backend Developer Agent
          │
          ├── Custom Agent(Coding Instructions)
          ├── API Development Skill
          ├── Testing Skill
          └── Architecture MCP
```

.NET Backend Developer Agent 實際結構

.NET Backend Developer Agent 在原始碼專案中會有以下的結構：

```
Git Repository root
├── .github/
│   ├── agents/
│   │   └── dotnet-backend-developer.agent.md
│   ├── skills/
│   │   ├── dotnet-api-design/
│   │   │   └── SKILL.md
│   │   ├── dotnet-architecture/
│   │   │   └── SKILL.md
│   │   ├── dotnet-observability-security/
│   │   │   └── SKILL.md
│   │   ├── dotnet-testing/
│   │   │   └── SKILL.md
│   │   └── ef-core-persistence/
```

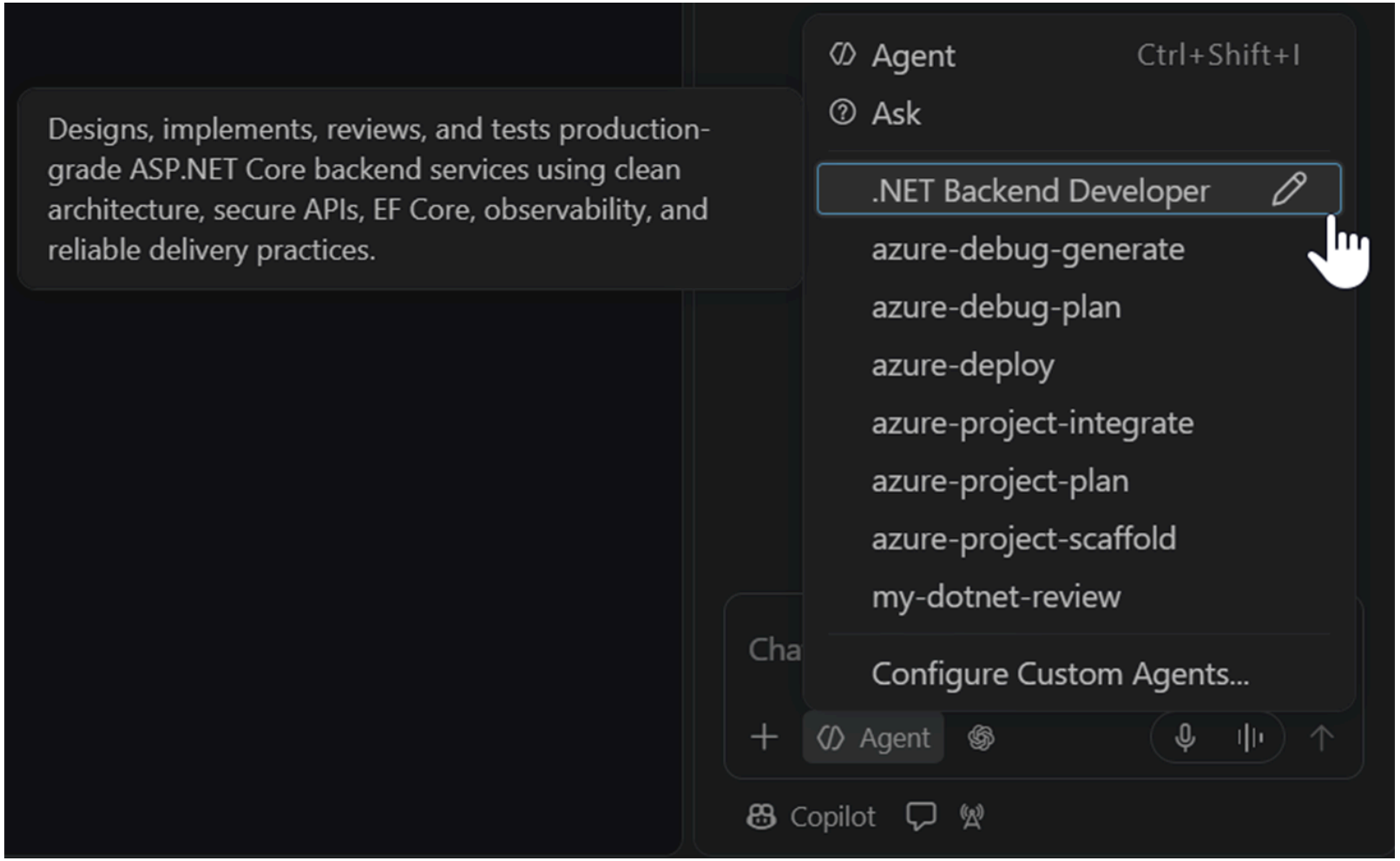
`dotnet-backend-developer.agent.md` 為 Custom Agent 的設定檔，定義了 Agent 的行為與使用的 Skills。

`skills/` 目錄下的每個子目錄對應一個 Skill，每個 Skill 內都有一個 `SKILL.md` 文件，定義了該 Skill 的行為與使用方式。

`.mcp.json` 為 MCP Server 的設定檔，定義 MCP Server 相關資訊與設定。

.NET Backend Developer Agent 運作流程

實際使用記得要先在 VS Code Chat View 中選取 .NET Backend Developer Agent：



Build Custom Agent Plugin

VSCode Agent Plugin

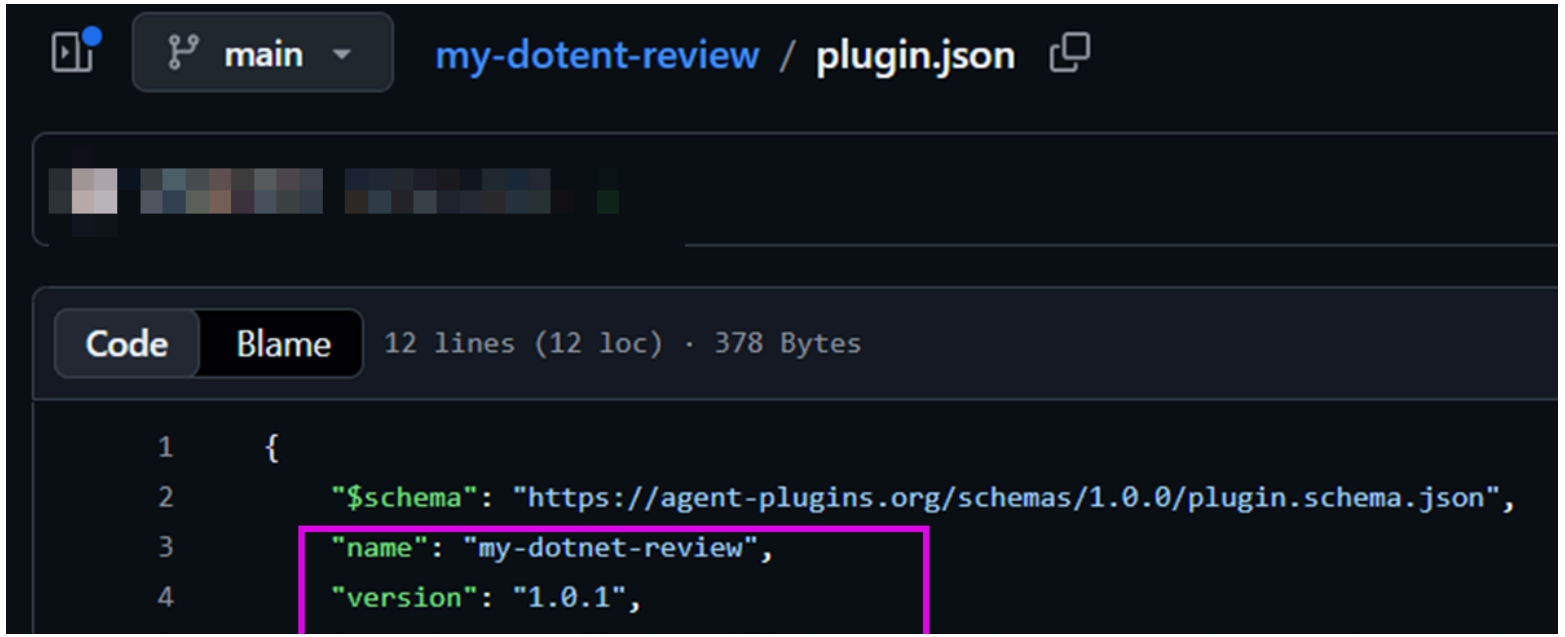
前面的 Custom Agent , Agent Skill, 以及 MCP Server 都是一堆很多文字&設定檔案，管理起來可能會比較麻煩，因此我們可以將它們打包成一個 VSCode Agent Plugin，方便在 VSCode 中使用與管理。

- 官方說明:
  - VSCode 官方說明：https://code.visualstudio.com/docs/agent-customization/agent-plugins
  - GitHub Copilot 官方說明: https://docs.github.com/en/copilot/concepts/customizations/plugins
- Agent Plugin 為一套目錄結構由『plugin.json』檔案所描述，可儲存:
  - MCP Server config
  - Custom Agent
  - Agent Skill
  - Automation Templates, Hook (Optional, not standard)

  相關資訊，並添加版本號碼方便管理與更新
- Plugin放在同電腦檔案目錄或是在 git repo 上的都可當安裝源

如何製作 VSCode Agent Plugin

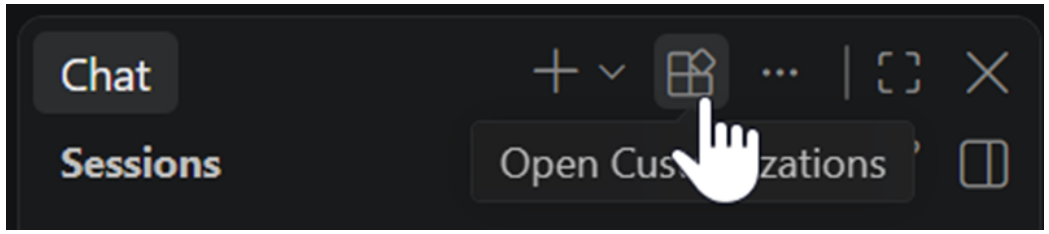
- `VSCode Agent Plugin 1.0` 規格遵循 ***Agent Plugins*** 公訂標準 (https://agent-plugins.org/)的目錄結構與設定規範
- 由一個 `plugin.json` 檔案描述(plugin manifest)：https://code.visualstudio.com/docs/agents/reference/agent-plugin-reference
- **plugin.json** 必填的欄位為 `$schema`, `name`, `version`, `description`， 其他如果都依照規格定義義之目錄放置相關檔案，會自動偵測到：



安裝 VSCode Agent Plugin

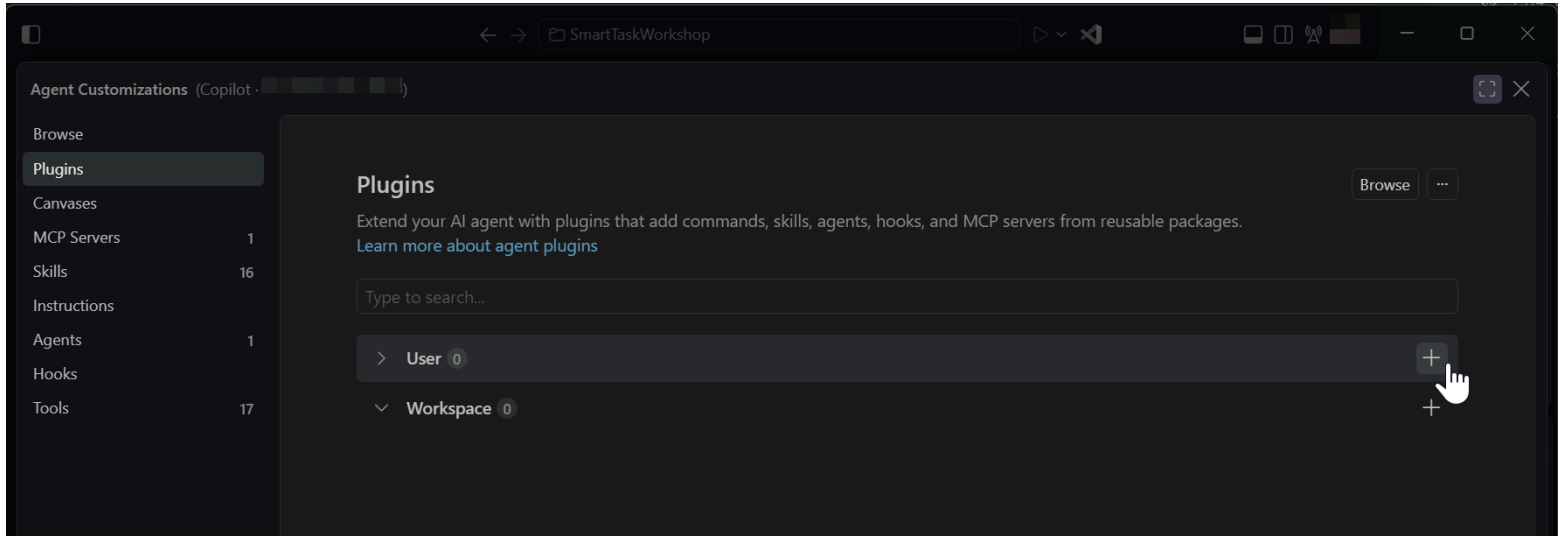
VSCode Agent Customization 提供了安裝 Agent Plugin 的功能，可以將本地或遠端的 Agent Plugin 安裝到 VSCode 中。

1. 打開 VSCode Agent Customization UI，



   左邊選擇 Plugins

2. 目前只能安裝在 User 層級，按右邊對應的『+』按鈕



結論

透過將 Custom Agent、Agent Skill 以及 MCP Server 打包成 VSCode Agent Plugin，可以更方便地在 VSCode 中管理與使用這些資源。無論是本地目錄還是遠端 Git repository 上的 Agent Plugin，都可以輕鬆安裝，並在 VSCode Agent Chat 或 Agent Window 中使用。這種方式不僅提升了管理效率，也方便了版本控制與更新。

> **Tip**
>
> 儘量都先以使用公訂規格的方向做客製化，以免在現在的 VSCode/GitHub Copilot 快速更版下，客製化功能就突然故障或得一直migrate